Fixing Low RAGAS Scores — Context Management Playbook

> **Symptom:** Faithfulness near 0 on most queries. Answer-relevancy moderate. Tool-precision moderate. Many queries log `0.0` across *all* metrics. **Root cause (in one line):** Contexts sent to RAGAS are truncated, mis-shaped, and sometimes empty — and when RAGAS errors the code silently writes `0.0` instead of skipping. Fix context plumbing first, then tune the answer shape and the judge.

---

1. Why so many queries score `0.0` on every metric

This is not a model-quality problem — it's a code path that swallows exceptions and stores defaults.

1.1 The silent-failure path in `evaluator.py`

```
faithfulness_score = 0.0
relevancy_score    = 0.0

try:
    result = evaluate(dataset, metrics=[faithfulness, answer_relevancy], ...)
    df = result.to_pandas()
    ...
except Exception as e:
    print(f"   ⚠  RAGAS evaluation error: {e}")
    print(f"   Using default scores (0.0) for this query")
```

`evaluation/evaluator.py:180-204`. Any exception inside `evaluate()` (rate limit, JSON-parse failure in the judge, missing OpenAI key, network blip, empty contexts list) → scores stay at `0.0` and the **point is still written to Influx**. That's why entire rows are zero.

Also, `raise_exceptions=False` (line 189) lets RAGAS *internally* return `NaN` per metric instead of raising. The `raw == raw` NaN-check on line 197 then coerces NaN → `0.0`. Two layers of "silently zero".

1.2 The empty-context path in `chatbot_api.py`

```
contexts = []
for step in result["steps"]:
    contexts.append(f"Tool: {step['tool']}\nResult: {step['result_preview']}")

if request.evaluate and contexts:
    eval_scores = evaluator.evaluate_answer(...)
```

`api/chatbot_api.py:67-85`. If the agent answered **without any tool call** (greeting, definition the model answered from memory, fallback summary), `steps == []` → `contexts == []` → evaluation is skipped → response has `eval_scores: {}` and **nothing is written** to Influx for that question.

So two failure shapes exist:

- **Zero row written** — RAGAS ran but errored / returned NaN.

- **No row at all** — agent never called a tool.

Both look the same on a Grafana dashboard ("metric is 0 / missing").

---

2. Why faithfulness specifically is near zero

Faithfulness = (supported claims) / (total claims). The judge can only mark a claim "supported" if it appears in `contexts`. The current pipeline starves the judge of evidence.

2.1 Contexts are truncated to 600 chars

`assistant/agent/orchestrator.py:288-291`:

```
preview = (
    tool_result[:600] + f"\n... [truncated, {len(tool_result)} chars total]"
    if len(tool_result) > 600 else tool_result
)
steps.append({..., "result_preview": preview, ...})
```

Then `chatbot_api.py:71` feeds **`result_preview`** to RAGAS — *not* the full tool output. So:

- The **model** saw the full `tool_result` (because the Part returned to Gemini contained the full string in `messages`).
- But **RAGAS** only sees the first 600 chars + a `[truncated]` marker.
- The model legitimately cites a value from char 800 of the sensor dump → RAGAS does not see that value → marks the claim unsupported → faithfulness drops.

This single bug accounts for most of the drift.

2.2 Contexts carry a wrapper string the judge interprets as content

```
Tool: fetch_realtime_sensors
Result: === BOILER REALTIME SENSORS ===
main_steam_temp_boiler = 542 °C ...
```

The `"Tool: …\nResult: …"` header inflates the context with metadata the model didn't claim, and the judge sometimes treats `"Tool: fetch_realtime_sensors"` as a claim source to verify *against* (it isn't).

2.3 The system prompt allows untracked sources

`orchestrator.py:74-79` says claims may come from "(a) tool result … or (b) explicit IBR knowledge you can state with certainty." Branch (b) means the model is free to insert engineering facts that won't appear in `contexts` → guaranteed unsupported by RAGAS, even when correct.

2.4 Knowledge-tool output uses similarity headers

`assistant/agent/tools/knowledge_tool.py:75-79` emits:

```
--- High CO in Chimney  (id=fault_high_co, category=fault, similarity=78.4%) ---
<doc text>
```

The model paraphrases the doc text in its answer. The judge sees the chunk and the doc text — usually fine — but if the doc itself was truncated by `result_preview` (Section 2.1) the supporting sentence is gone.

2.5 Judge LLM (Gemini Flash) is the cheapest tier

Flash is fast but misses paraphrase support. A claim like "Feedwater pressure is 17.2 MPa, below the normal 18.0–20.0 MPa band" is supported by a context that lists `feedwater_pressure = 17.2 MPa, status = WARNING, normal 18-20 MPa`, but Flash sometimes scores it as unsupported because the wording shifts.

---

3. Why answer relevancy is only moderate

Relevancy = mean cosine similarity between the user's question and N synthetic questions reverse-generated from the **answer**.

- The system prompt encourages a 5-section template (`Current Status / Diagnosis / Root Cause / Immediate Actions / Prevention`) for fault questions. When applied to short questions, the answer covers ground the question didn't ask about → reverse-generated questions drift → cosine drops.
- Answers often open with boilerplate ("Based on the current sensor readings…") before answering. The reverse-question generator latches onto the boilerplate.
- For multi-intent questions the answer is long, so the embedding of any single reverse-question is generic.

---

4. Why tool precision is only moderate

`evaluator.py::EXPECTED_TOOLS_MAP` is a keyword bag (`"current"` → `fetch_realtime_sensors`, `"why"` → `search_knowledge_base`, …).

- Keywords overlap across intents ("why is the **current** value high" matches both `current` and `why`).
- Default falls back to `fetch_realtime_sensors` when no keyword matches → falsely expected.
- Multi-intent questions inflate the expected set; if the agent (correctly) skipped a tool, precision is punished.

This is a metric problem, not an agent problem. Keep the weight low (0.2) and don't over-index on it.

---

5. Fix Plan — Context Management First

Apply in this order. Each step is verifiable in Grafana before moving on.

5.1 [TOP FIX] Stop sending truncated previews to RAGAS

Change orchestrator — store the full tool result, keep the preview only for the UI.

```
# assistant/agent/orchestrator.py — inside the tool loop
steps.append({
    "step":           step_count,
    "tool":           tool_name,
    "args":           tool_args,
    "result":         tool_result,           # NEW — full content
    "result_preview": preview,               # keep for UI / logs
    "result_length":  len(tool_result),
})
```

Change API — build contexts from `result`, not `result_preview`.

```
# api/chatbot_api.py
contexts = [step["result"] for step in result["steps"]]
tools_called = [step["tool"] for step in result["steps"]]
```

Drop the `"Tool: X\nResult: Y"` wrapper — pass each tool output as its own context string. The judge then has clean evidence per document.

Expected impact: **faithfulness +0.3 to +0.5** on questions whose tool output exceeded 600 chars (most of them).

5.2 Chunk large tool outputs

Some tool outputs (especially `search_knowledge_base` with `top_k=5`) can be 3–5 KB. RAGAS judges each "context" as one document; very long contexts confuse claim verification.

Split into one context per logical block:

```
def split_contexts(tool_name: str, raw: str) -> list[str]:
    if tool_name == "search_knowledge_base":
        # Split on the "--- <title> ---" separator
        chunks = re.split(r"\n--- .*? ---\n", raw)
        return [c.strip() for c in chunks if c.strip()]
    if tool_name == "fetch_realtime_sensors":
        # One context per sensor group (BOILER, TURBINE, CHIMNEY)
        return [block.strip() for block in raw.split("\n\n") if block.strip()]
    return [raw]

contexts = []
for s in result["steps"]:
    contexts.extend(split_contexts(s["tool"], s["result"]))
```

Expected impact: **faithfulness +0.05 to +0.1**, fewer false-unsupported verdicts on long docs.

5.3 Stop writing `0.0` rows when RAGAS fails

Replace the silent-fail block:

```
try:
    result = evaluate(...)
    df = result.to_pandas()
    raw_faith = df["faithfulness"].iloc[0]
    raw_relev = df["answer_relevancy"].iloc[0]

    faithfulness_score = None if raw_faith != raw_faith else
```

```
round(float(raw_faith), 3)
    relevancy_score    = None if raw_relev != raw_relev else
round(float(raw_relev), 3)

except Exception as e:
    print(f"   ⚠  RAGAS evaluation error: {e}")
    faithfulness_score = None
    relevancy_score    = None
```

Then in `_log_to_influx`, **skip the metric field if the value is None** instead of writing `0.0`. Add a tag `eval_status="ok"` or `"failed"` so Grafana can plot success-rate separately from quality.

Expected impact: dashboards become **truthful**. The "lots of zeros" panel goes away; you see real failures vs real low scores.

5.4 Always evaluate, even when no tool was called

```
# api/chatbot_api.py
if request.evaluate:
    contexts = contexts or [
        "(agent answered without calling any tool — no retrieved context)"
    ]
    eval_scores = evaluator.evaluate_answer(...)
```

For tool-free answers this will (correctly) score faithfulness very low if the answer makes factual claims, or 1.0 if the answer is a greeting / refusal. Either way you get a metric instead of a gap.

Mark these rows with a tag `had_tool_call=false` so you can filter them.

5.5 Use the same judge model RAGAS expects

Replace Flash with Pro for the judge only. Keep the agent on Flash for cost.

```
self.judge = ChatVertexAI(
    model_name="gemini-2.5-pro",
    project=GCP_PROJECT_ID,
    location=GCP_REGION,
    temperature=0,
)
```

Expected impact: **faithfulness +0.05 to +0.15** (paraphrase support gets recognised), **answer_relevancy +0.05** (better reverse-Q generation).

5.6 Constrain the agent to ground every claim in tool output

Edit the system prompt in `orchestrator.py:70-187`:

- Remove branch (b) ("explicit IBR knowledge you can state with certainty").
- Add: *"Every factual sentence must paraphrase a phrase that appears in a tool result. If a tool returned no relevant data, say so — do not fill in engineering knowledge from training."*
- Add: *"Open every answer with one sentence that directly answers the user's question using the same key terms they used."*

This both helps faithfulness (fewer ungrounded claims) and relevancy (first sentence anchors the reverse-Q to the user's wording).

5.7 Match answer length to question shape

In the system prompt, replace the soft "shape the answer to the question" with hard caps:

```
- Definition / "what is X"          → ≤4 sentences, no sections.
- Single sensor value               → 1 sentence: value, unit, status, normal
band.
- Yes/no question                   → answer first sentence, evidence second.
- Trend / prediction                → 3 sentences max.
- Fault diagnosis                   → up to 5 sections, but skip empty ones.
- History summary                   → bullets, only what get_fault_history
returned.
```

Short, targeted answers improve **relevancy** (less drift in reverse-Qs) and **faithfulness** (fewer unsupported sentences).

5.8 Replace tool-precision keyword bag with intent classification (optional)

If §5.1-§5.7 land you in a good place on F & R, then revisit tool precision. Either:

- Use a small LLM call (cached) to label the question's intents (`present/knowledge/past/future`) and map intents → expected tools. Removes the keyword overlap problem.
- Or weight by recall not precision: `|called ∩ expected| / |called ∪ expected|` (Jaccard). Penalises both extra and missed tools symmetrically.

5.9 Audit the knowledge base

If `search_knowledge_base` is the dominant tool but `similarity < 70%` consistently, the KB is the bottleneck:

```
python -m knowledge_base.indexer --mode=verify
```

Cross-check the docs against `query.csv` queries. Add missing fault codes, IBR clauses, multi-sensor patterns. Re-index with `--mode=full`.

---

6. Concrete File-by-File Diff Summary

File	Change	Section
`assistant/agent/orchestrator.py`	Store full `tool_result` in `steps`, keep preview separately. Tighten system prompt (5.6, 5.7).	5.1, 5.6, 5.7
`api/chatbot_api.py`	Build `contexts` from `step["result"]`, not preview. Always evaluate.	5.1, 5.2, 5.4

File	Change	Section
	Chunk by tool.	
`evaluation/evaluator.py`	Return `None` (not `0.0`) on failure. Skip None fields when writing Influx. Add `eval_status`, `had_tool_call` tags. Switch judge to `gemini-2.5-pro`. Add intent-based or Jaccard tool-precision.	5.3, 5.5, 5.8
`knowledge_base/boiler_guide.py`	Add missing docs uncovered by audit. Re-index.	5.9

---

7. Verification Plan

After **each** change in §5, re-run a fixed query set and compare distributions.

7.1 Reproducible benchmark

```
# 1. Run the full query set
python -m evaluation.batch_eval --queries query.csv --out runs/run-$(date +
%s).jsonl
```

(If no batch runner exists, write a tiny script that POSTs every row in `query.csv` to `/chat` and stores the response JSON.)

7.2 Grafana panels to add

Use the `chatbot_evaluation` measurement.

A. Eval success rate — counts of `eval_status` per hour:

```
from(bucket: "boiler_data")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "chatbot_evaluation")
  |> filter(fn: (r) => r._field == "faithfulness")
  |> group(columns: ["eval_status"])
  |> aggregateWindow(every: 1h, fn: count, createEmpty: true)
```

B. Per-metric distribution — histogram of faithfulness values, excluding failed evals:

```
from(bucket: "boiler_data")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "chatbot_evaluation")
  |> filter(fn: (r) => r._field == "faithfulness")
  |> filter(fn: (r) => r.eval_status == "ok")
  |> histogram(bins: [0.0, 0.2, 0.4, 0.6, 0.8, 1.0])
```

C. Worst questions — table of lowest faithfulness:

```
from(bucket: "boiler_data")
```

```
  |> range(start: -7d)
  |> filter(fn: (r) => r._measurement == "chatbot_evaluation")
  |> filter(fn: (r) => r._field == "faithfulness")
  |> filter(fn: (r) => r._value < 0.5)
  |> keep(columns: ["_time", "question_preview", "_value"])
  |> sort(columns: ["_value"], desc: false)
  |> limit(n: 25)
```

7.3 Expected progression

Stage	Expected faithfulness median	Notes
Baseline (now)	0.10–0.30	Many `0.0` rows from silent failures
After §5.1 (full contexts)	0.55–0.70	Biggest jump
After §5.2 (chunking)	0.60–0.75	Cleaner judge decisions
After §5.3 (no zero-rows)	unchanged median, but distribution becomes interpretable	
After §5.5 (Pro judge)	0.65–0.80	Paraphrase support recognised
After §5.6+§5.7 (prompt)	0.75–0.85	Fewer ungrounded sentences

Answer relevancy should track from moderate (≈0.5–0.65) up to **0.75–0.85** after §5.6+§5.7.

---

8. Quick Sanity Checklist Before Calling It Fixed

- ☐ No more `eval_scores: {}` returned by `/chat` (always populated or marked failed).
- ☐ No `0.0` rows in Influx unless RAGAS actually said `0.0` (verifiable by checking `eval_status` tag).
- ☐ `contexts` length matches `tool_result_length` of all steps (no silent truncation).
- ☐ Judge model is `gemini-2.5-pro`, not Flash, in `evaluation/evaluator.py`.
- ☐ System prompt forbids ungrounded engineering claims.
- ☐ Knowledge-base verify reports ≥ expected document count.
- ☐ Re-run of `query.csv` shows faithfulness median ≥ 0.7 and answer-relevancy median ≥ 0.75.
- ☐ Grafana dashboard distinguishes `eval_status=ok` vs `failed` rows.

---

9. One-Paragraph Summary (for the meeting)

Faithfulness is collapsing because the evaluator only sees a 600-character truncated preview of each tool's output, while the model answers using the full output — every claim past char 600 looks

unsupported. On top of that, the evaluator silently writes `0.0` whenever RAGAS errors, which is why so many rows are zero across all metrics. Fix is in this order: (1) feed RAGAS the **full** tool result, not the preview; (2) chunk long tool outputs into per-document contexts; (3) return `None` (skip the field) on RAGAS failure instead of `0.0`; (4) always evaluate (even for tool-free answers, with a marker tag); (5) upgrade the judge LLM from Gemini Flash to Pro; (6) tighten the system prompt to ground every claim in a tool result and forbid template padding. Verify each step against `query.csv` and the Grafana distribution panel before moving on.